SOFTWARE REQUIREMENTS & DESIGN SPECIFICATION

BidBot

Supervised By

Dr. Asif Naeem

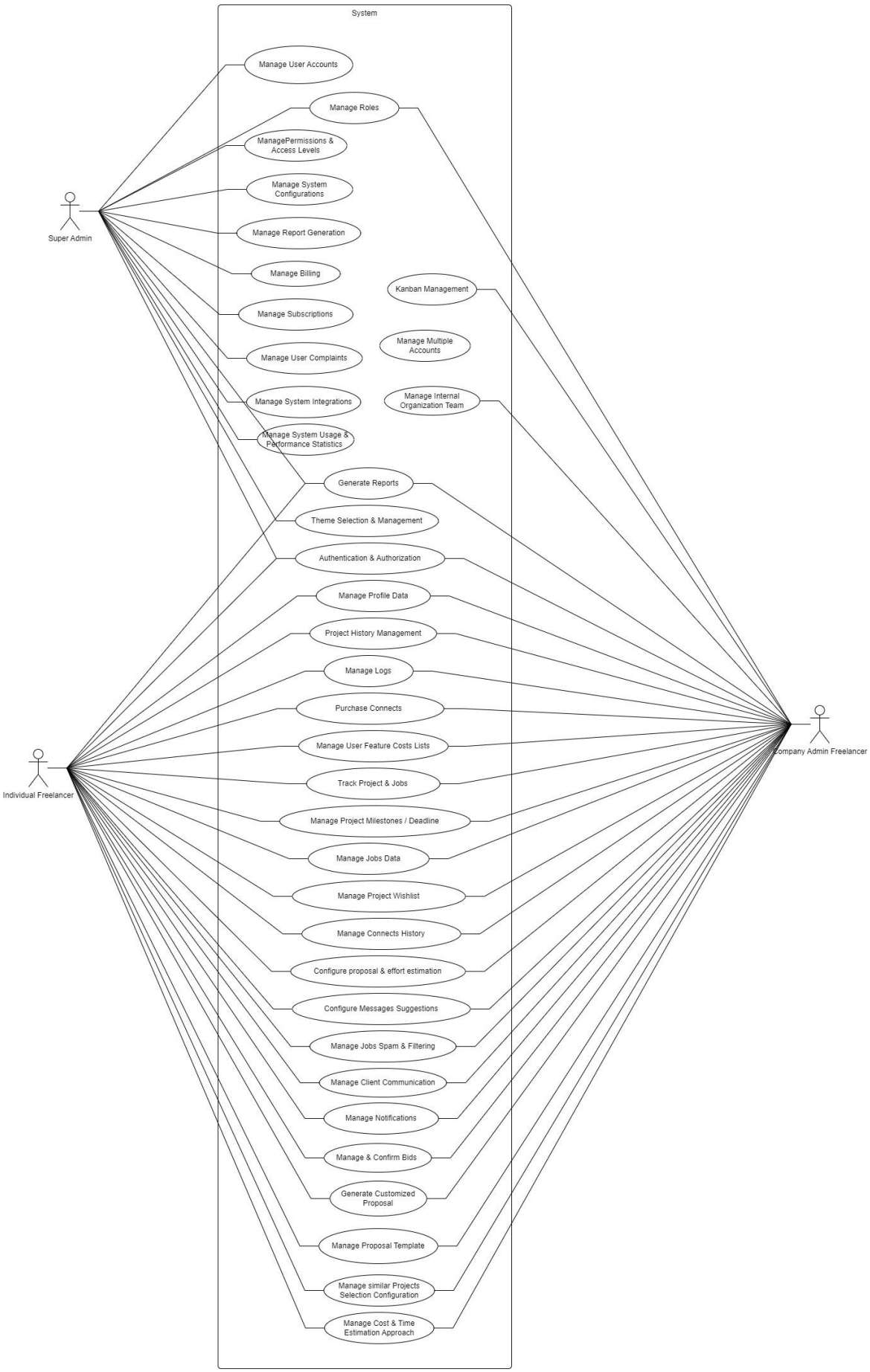
Team Members

Basit Nazir

Muhammad Bilal

Table of Contents

Use Case Diagram



High Level Use Cases

High level use case: User Management	
Use case	Manage User Accounts
Actor	Super Admin
Type	Primary
Description	The Super Admin manages individual freelancer and company accounts, including creating, updating, and deleting user profiles.

High level use case: User Management	
Use case	Set Permissions
Actor	Super Admin
Type	Primary
Description	The Super Admin sets permissions and access levels for different user roles within the platform.

High level use case: System Configuration	
Use case	Configure System Settings
Actor	Super Admin
Type	Primary
Description	The Super Admin configures system settings and integrations to optimize platform performance.

High level use case: System Configuration	
Use case	Manage Subscription Plans
Actor	Super Admin
Type	Primary
Description	The Super Admin manages various subscription plans available to users, including pricing and features.

High level use case: Subscriptions and Billing	
Use case	Manage Subscriptions
Actor	Super Admin

Type	Primary
Description	The Super Admin oversees the management of user subscriptions, including upgrades and downgrades.

High level use case: Subscriptions and Billing	
Use case	Manage Billing
Actor	Super Admin
Type	Primary
Description	The Super Admin handles all billing-related activities, ensuring accurate invoicing and payments.

High level use case: Subscriptions and Billing	
Use case	Generate Bills Report
Actor	Super Admin
Type	Primary
Description	The Super Admin generates reports on billing activities for analysis and record-keeping.

High level use case: Subscriptions and Billing	
Use case	Manage Payment Gateway
Actor	Super Admin
Type	Primary
Description	The Super Admin manages the payment gateway integration, ensuring secure transactions.

High level use case: Complaints System	
Use case	Manage User Complaints
Actor	Super Admin
Type	Primary
Description	The Super Admin manages user complaints, including tracking, resolving, and reporting on issues raised by users.

High level use case: Analytics, Logging, and Reporting

Use case	View Usage Statistics
Actor	Super Admin
Type	Primary
Description	The Super Admin views system-wide usage statistics and performance metrics to gauge platform effectiveness.

High level use case: Analytics, Logging, and Reporting	
Use case	Generate Activity Reports
Actor	Super Admin
Type	Primary
Description	The Super Admin generates reports on user activities and system performance for analysis and improvements.

High level use case: Theme Selection and Management	
Use case	Manage Theme Selection
Actor	Super Admin
Type	Primary
Description	The Super Admin selects and manages themes for the platform to enhance user experience and branding.

High level use case: Authentication and Authorization	
Use case	Two-Factor Authentication
Actor	Freelancer
Type	Primary
Description	The freelancer enables two-factor authentication to enhance account security.

High level use case: Authentication and Authorization	
Use case	JWT and Session Tokens
Actor	Freelancer
Type	Primary
Description	The freelancer uses JWT (Web Tokens) and session tokens for secure authentication during logins.

High level use case: Profile Management	
Use case	Manage Projects History
Actor	Freelancer
Type	Primary
Description	The freelancer manages their project history, including viewing features and prices associated with past projects.

High level use case: Profile Management	
Use case	Manage Personal Data
Actor	Freelancer
Type	Primary
Description	The freelancer updates and maintains personal data on their profile.

High level use case: Profile Management	
Use case	Manage Feature Costs List
Actor	Freelancer
Type	Primary
Description	The freelancer manages a list of costs associated with various features offered in their services.

High level use case: Connects Purchasing Module	
Use case	Execute Purchasing Workflow
Actor	Freelancer
Type	Primary
Description	The freelancer initiates and completes the workflow to purchase connects.

High level use case: Connects Purchasing Module	
Use case	Manage Payment Medium
Actor	Freelancer
Type	Primary

Description	The freelancer manages payment methods for purchasing connects.

High level use case: Connects Purchasing Module	
Use case	View Connects History
Actor	Freelancer
Type	Primary
Description	The freelancer views their history of purchased connects.

High level use case: Project / Jobs Tracking Module	
Use case	Manage Ongoing Projects
Actor	Freelancer
Type	Primary
Description	The freelancer views and manages ongoing projects, including deadlines and milestones.

High level use case: Project / Jobs Tracking Module	
Use case	Manage Wish-list
Actor	Freelancer
Type	Primary
Description	The freelancer manages a wish-list of desired projects or jobs.

High level use case: Proposal and Effort Estimation Configuration	
Use case	Configure Proposal Settings
Actor	Freelancer
Type	Primary
Description	The freelancer customizes settings for proposal generation and effort estimation.

High level use case: Messages Suggestions Configuration	
Use case	Configure Message Suggestions
Actor	Freelancer
Type	Primary

Description	The freelancer configures message suggestions for client communications.

High level use case: Job Filtering and Spam Detection Configuration	
Use case	Manage Spam Detection Parameters
Actor	Freelancer
Type	Primary
Description	The freelancer configures parameters for spam detection and job filtering.

High level use case: Job Filtering and Spam Detection Configuration	
Use case	Manage Spam Keywords
Actor	Freelancer
Type	Primary
Description	The freelancer manages the list of keywords used for identifying spam job postings.

High level use case: Client Communication Management	
Use case	Manage Client Communication
Actor	Freelancer
Type	Primary
Description	The freelancer communicates with clients regarding projects and proposals.

High level use case: Notifications and Alerts Module	
Use case	Manage Notifications
Actor	Freelancer
Type	Primary
Description	The freelancer receives and manages notifications and alerts related to projects and proposals.

High level use case: Reports Generation Module	
Use case	Generate Reports
Actor	Freelancer
Type	Primary
Description	The freelancer generates reports on project activities, earnings, and other relevant metrics.

High level use case: Dashboard and Analytics Module	
Use case	View Dashboard Analytics
Actor	Freelancer
Type	Primary
Description	The freelancer views analytics related to their projects and overall dashboard performance.

High level use case: Bids Confirmation and Management	
Use case	Manage Bids History
Actor	Freelancer
Type	Primary
Description	The freelancer views and manages the history of their submitted bids.

High level use case: Help and Support Module	
Use case	Access Help and Support
Actor	Freelancer
Type	Primary
Description	The freelancer accesses help and support resources for any issues or inquiries.

High level use case: Multi-Account Management	
Use case	Manage Multiple Upwork Accounts
Actor	Company Admin
Type	Primary

Description	The Company Admin manages multiple Upwork accounts from a single dashboard interface.

High level use case: Multi-Account Management	
Use case	Switch Accounts
Actor	Company Admin
Type	Primary
Description	The Company Admin switches between different Upwork accounts for monitoring and management purposes.

High level use case: Project Manager Module	
Use case	Create and Manage Projects
Actor	Project Manager
Type	Primary
Description	The Project Manager creates and manages projects, including setting deadlines and assigning tasks.

High level use case: Project Manager Module	
Use case	Track Project Progress
Actor	Project Manager
Type	Primary
Description	The Project Manager tracks the progress of ongoing projects and updates statuses as needed.

High level use case: CTO/Admin Module	
Use case	Oversee Company Operations
Actor	CTO/Admin
Type	Primary
Description	The CTO/Admin oversees overall company operations and project management across multiple accounts.

High level use case: CTO/Admin Module	
Use case	Generate Reports

Actor	CTO/Admin
Type	Primary
Description	The CTO/Admin generates reports on project performance, team productivity, and financial metrics.

High level use case: Sales / Bidding Team Module	
Use case	Manage Bids
Actor	Sales/Bidding Team Member
Type	Primary
Description	The Sales/Bidding Team Member creates, submits, and manages bids for projects on behalf of the company.

High level use case: Sales / Bidding Team Module	
Use case	Track Bid Status
Actor	Sales/Bidding Team Member
Type	Primary
Description	The Sales/Bidding Team Member tracks the status of submitted bids and follows up as necessary.

High level use case: Internal Organization Team Management	
Use case	Organize Internal Teams
Actor	Company Admin
Type	Primary
Description	The Company Admin organizes internal teams based on project needs and skills.

High level use case: Internal Organization Team Management	
Use case	Assign Roles
Actor	Company Admin
Type	Primary
Description	The Company Admin assigns roles to team members for various projects and tasks.

High level use case: Permissions and Access Level Management	
Use case	Manage Permissions
Actor	Company Admin
Type	Primary
Description	The Company Admin manages permissions and access levels for different team members and roles.

High level use case: Internal Organization Role Management	
Use case	Define Internal Roles
Actor	Company Admin
Type	Primary
Description	The Company Admin defines and manages internal roles within the organization to ensure proper function and authority.

High level use case: Kanban Module	
Use case	Manage Kanban Boards
Actor	Company Admin
Type	Primary
Description	The Project Manager or team member manages Kanban boards to visualize and organize project tasks and workflows.

High level use case: Kanban Module	
Use case	Update Task Status
Actor	Project Manager / Team Member
Type	Primary
Description	The Project Manager or team member updates the status of tasks on the Kanban board to reflect current progress.

High level use case: Configured / Customized Proposal Generation	
Use case	Configure Proposal Settings

Actor	Freelancer / Company User
Type	Primary
Description	The user configures settings for customized proposal generation, specifying preferences and requirements.

High level use case: Configured / Customized Proposal Generation	
Use case	Generate Proposal
Actor	Freelancer / Company User
Type	Primary
Description	The user generates a proposal based on the configured settings and content templates.

High level use case: Document Content Extraction	
Use case	Extract Content from Documents
Actor	Freelancer / Company User
Type	Primary
Description	The user extracts relevant content from uploaded documents to include in the proposal.

High level use case: Document Content Extraction	
Use case	Review Extracted Content
Actor	Freelancer / Company User
Type	Primary
Description	The user reviews the extracted content for accuracy and relevance before finalizing the proposal.

High level use case: Proposal Template Management	
Use case	Manage Proposal Templates
Actor	Freelancer / Company User
Type	Primary
Description	The user creates, edits, and deletes proposal templates to streamline the proposal generation process.

High level use case: Proposal Template Management	
Use case	Select Template for Proposal
Actor	Freelancer / Company User
Type	Primary
Description	The user selects an appropriate template for the proposal based on project type and client preferences.

High level use case: Modules Breakdown with Time	
Use case	Define Modules Breakdown
Actor	Freelancer / Company User
Type	Primary
Description	The user defines the breakdown of project modules and associated time estimates for each module in the proposal.

High level use case:	
Use case	Update Time Estimates
Actor	Freelancer / Company User
Type	Primary
Description	The user updates time estimates for modules as needed based on project scope changes.

High level use case:	
Use case	Search for Similar Projects
Actor	Freelancer / Company User
Type	Primary
Description	The user searches for and selects previous similar projects to reference in the proposal.

High level use case:	
Use case	Preview Selected Projects
Actor	Freelancer / Company User
Type	Primary

Description	The user previews selected previous projects to ensure they align with the current proposal’s context.

High level use case:	
Use case	Calculate Project Cost and Time
Actor	Project Manager / Freelancer
Type	Primary
Description	The user calculates the estimated cost and time for a project by inputting project details and selecting predefined configurations. The module supports a breakdown of project modules and allows for estimations based on the client's budget and specific requirements.

High level use case:	
Use case	Calculate Project Cost and Time
Actor	Project Manager / Freelancer
Type	Primary
Description	The user calculates the estimated cost and time for a project by inputting project details and selecting predefined configurations. The module supports a breakdown of project modules and allows for estimations based on the client's budget and specific requirements.

Functional Requirements

Super Admin User Management Module

- The system shall allow the Super Admin to create, read, update, and delete (CRUD) individual freelancer and company accounts.
- The system shall provide a searchable list of all freelancer and company accounts.
- The system shall allow the Super Admin to view detailed information about each account, including profile details and current subscription status.
- The system shall allow the Super Admin to suspend(block) or reactivate (unblock) user accounts.
- The system shall enable the Super Admin to assign different access levels / permissions to freelancers and companies.
- The system shall allow the creation of predefined roles (e.g., Super Admin, Company admin, Individual Freelancer, Company Freelancer) with specific permissions.
- The system shall enable the Super Admin to customize permissions for each user.
- The system shall automatically enforce permissions based on the assigned role and configured permissions of each user when they log in.
- The system shall allow the Super Admin to assign roles (e.g., Super Admin, Company admin, Individual Freelancer, Company Freelancer) to newly created accounts.
- The system shall allow modification of user roles after account creation.
- The system shall notify users when their account status changes (e.g., suspension, reactivation).
- The system shall enforce strong password requirements for all user accounts.
- The system shall allow the Super Admin to reset user passwords.
- The system shall support two-factor authentication (2FA) for Super Admin access to enhance security.

Super Admin System Configuration Module

- The system shall allow the Super Admin to create, edit, and delete subscription plans with different pricing tiers and features.
- The system shall allow the Super Admin to define subscription plan details, including pricing, billing cycle (monthly/annually), and available features.
- The system shall enable the Super Admin to set up discounts or promotional codes for specific subscription plans.
- The system shall allow the Super Admin to update or modify subscription plans without affecting active subscriptions.
- The system shall provide a feature for the Super Admin to view all active and expired subscriptions for users.
- The system shall allow the Super Admin to configure security-related settings, including password policies (e.g., length, complexity).
- The system shall enable the Super Admin to manage authentication settings, including enabling or disabling two-factor authentication (2FA) for users.
- The system shall allow the Super Admin to manage session timeout settings and login attempt limits.

Super Admin Subscriptions and Billing Module

- The system shall allow the Super Admin to create, modify, and delete subscription plans with varying tiers (e.g., basic, premium, enterprise) and durations (monthly, annually).
- The system shall allow the Super Admin to assign specific features and limitations to each subscription plan.

- The system shall enable the Super Admin to view a list of active, expired, and cancelled subscriptions.
- The system shall provide the ability to upgrade, downgrade, or cancel subscriptions for individual users or companies.
- The system shall send automated notifications to users regarding upcoming renewals, expiration, or changes in their subscription status.
- The system shall allow the Super Admin to manually activate or deactivate subscriptions.
- The system shall enable the Super Admin to view and manage billing cycles (e.g., monthly, yearly) for all users.
- The system shall allow the Super Admin to view detailed billing history for each user or company (Account).
- The system shall automatically generate billing reports detailing user payments.
- The system shall generate summaries of total income, and payment status for each subscription tier.
- The system shall allow the Super Admin to export billing reports in multiple formats (e.g., PDF, CSV).
- The system shall send automated email notifications to users for actions like:
    - Payment receipts.
    - Failed payment attempts.
    - Upcoming invoice due dates.
    - Subscription renewals or cancellations.

Super Admin Complaints System Module

- The system shall allow users (freelancers and companies) to submit complaints via a designated complaint form.
- The system shall capture complaint details such as the subject, description, category (e.g., billing, account issues).
- The system shall provide users with a reference number for each submitted complaint for tracking purposes.
- The system shall allow users to attach relevant documents or screenshots when submitting a complaint.
- The system shall automatically categorize complaints based on predefined categories (e.g., account management, billing, technical issues).
- The system shall allow the Super Admin to create, edit, and delete complaint categories.
- The system shall enable the Super Admin to track the status of each complaint (e.g., submitted, in-progress, resolved, closed).
- The system shall allow the Super Admin to update the status of complaints and add notes or comments for internal tracking.
- The system shall notify the user via email when the status of their complaint changes (e.g., received, resolved).
- The system shall provide a view for users to track the status and history of their submitted complaints.
- The system shall allow the Super Admin to resolve complaints and provide a resolution description.
- The system shall allow the Super Admin to close complaints once resolved and notify the user of the resolution.
- The system shall automatically send email notifications to users when:

    - Their complaint is received.
    - The complaint is being worked on or assigned.
    - The complaint is resolved or closed.

Super Admin Analytics, Logging, and Reporting Module

- The system shall provide real-time data on system-wide usage statistics, including:
    - Total number of active subscriptions by plan type.
- The system shall allow the Super Admin to view detailed usage statistics, such as the number of jobs applied, proposals submitted, and connects purchased.
- The system shall display graphical representations (e.g., charts, graphs) of usage trends over time (daily, weekly, monthly).
- The system shall allow the Super Admin to filter and view specific usage statistics for individual users or companies.
- The system shall log all key user activities, including:
    - User logins and logouts.
    - Subscription changes (e.g., upgrades, downgrades, cancellations).
    - Job application and proposal submission history.
    - Complaints submissions and resolutions.
- The system shall maintain a searchable log of all user activities for auditing purposes.

Freelancer Authentication and Authorization Module

- The system shall allow users to authenticate using their email and password.
- The system shall implement password strength validation during registration, requiring a mix of uppercase, lowercase, numbers, and special characters.
- The system shall allow users to reset their password via an email-based password recovery process.
- The system shall support two-factor authentication (2FA) to enhance security during user login.
- The system shall allow users to enable or disable 2FA from their profile settings.
- The system shall send a one-time password (OTP) via email when 2FA is enabled.
- The system shall allow users to regenerate the OTP if it is not received within a specific time frame.
- The system shall notify the user via email if there is an failed login attempt.
- The system shall use JWT to handle authentication in a stateless manner for API-based requests.
- The system shall generate a JWT upon successful user login and attach it to the user's session for subsequent API calls.
- The system shall implement token expiration for JWT (e.g., 24 hours), after which the user must re-authenticate.
- The system shall assign specific roles to users, such as "Freelancer" and "Super Admin," with different permissions for accessing certain features.
- The system shall restrict access to certain pages or functionalities based on the user’s role.
- The system shall check user permissions before allowing actions such as project management, proposal submission, and profile updates.
- The system shall provide an error message or redirect users if they attempt to access unauthorized pages or features.
- The system shall allow users to log in from multiple devices (e.g., mobile app, web browser) simultaneously, while securely managing sessions.

Freelancer Profile Management Module

- The system shall allow users to add, edit, and delete entries in their project history, including project names, descriptions, and associated features.
- The system shall enable users to manage project features, including:
    - Feature name.
    - Feature description.
    - Feature cost (price) for each feature provided in the project.
- The system shall provide users the ability to upload attachments (e.g., images, PDFs) related to project deliverables or documentation.
- The system shall display the total project cost, calculated by aggregating the prices of the individual project features.
- The system shall allow users to filter and search for specific projects based on name, feature, or date.
- The system shall allow users to view past project details, including features and associated pricing, in a user-friendly layout.
- The system shall allow users to view and update their personal information, including:
    - Name.
    - Profile picture.
    - Email address.
    - Phone number.
    - Professional bio.
- The system shall validate updated personal data, such as ensuring a valid email format and phone number.
- The system shall allow users to change their password, requiring both the current password and the new password.
- The system shall notify users via email or SMS when personal information (such as email or phone number) is updated.
- The system shall allow users to manage a list of frequently used project features and their associated costs.
- The system shall allow users to add, edit, and remove features in their feature cost list.
- The system shall store feature details, including:
    - Feature name.
    - Feature description.
    - Cost (price) for each feature.
- The system shall display the total value of all features in the feature cost list.
- The system shall support data encryption for sensitive user data (e.g., passwords)
- The system shall provide a detailed log of profile updates, including the time and date of changes made.

Freelancer Connects Purchasing Module

- The system shall allow users to initiate the purchasing process for connects directly from their connects page.
- The system shall provide users with a user-friendly interface to select the number of connects they wish to purchase.
- The system shall implement a step-by-step purchasing workflow, including:
    - Selecting connects.
    - Choosing a payment method.

- The system shall provide confirmation to the user upon successful completion of the purchase, including transaction details and the updated balance of connects available.
- The system shall notify users of any errors during the purchasing process (e.g., insufficient funds, payment failure) with clear messages .
- The system shall allow users to manage their preferred payment methods, including:
    - Credit/Debit cards.
    - PayPal.
- The system shall require users to enter valid payment information, including card number, expiration date, CVV, and billing address for card transactions.
- The system shall allow users to save their payment methods securely for future transactions.
- The system shall provide users with an option to select their preferred payment method at the time of purchase.
- The system shall maintain a detailed history of all connects purchases made by the user, including:
    - Purchase date.
    - Number of connects purchased.
    - Total cost of the purchase.
    - Payment method used.
    - Transaction status (e.g., completed, pending, failed).
- The system shall provide users with a dedicated section in their dashboard to view their connects history.
- The system shall allow users to filter and search through their connects history based on date ranges, transaction status, or payment method.
- The system shall provide users with the ability to download their connects purchase history as a CSV or PDF report for record-keeping purposes.
- The system shall display real-time updates of the user's connects balance after each purchase.
- The system shall provide users with analytics on their connects usage, including:
    - Total connects purchased over a specified period.
    - Average spending on connects per month.
    - Trends in connects purchases based on user activity.

Freelancer Project / Jobs Tracking Module

- The system shall provide users with an overview of all ongoing projects in a dedicated section of their dashboard.
- The system shall display the following information for each ongoing project:
    - Project title.
    - Client name.
    - Project status (e.g., In Progress, Completed, On Hold).
    - Start date and end date.
    - Deadline for each milestone.
    - Total budget and payments received.
- The system shall allow users to click on a project to view detailed information, including project descriptions, tasks, and communications with clients.
- The system shall enable users to update project status and progress, including marking tasks as completed or updating milestones.
- The system shall allow users to set and edit deadlines for projects and milestones

- The system shall send notifications or reminders to users for important project deadlines or milestone achievements via email or in-app alerts.
- The system shall allow users to create, edit, and delete milestones associated with each project.
- The system shall enable users to set milestone deadlines and track their completion status.
- The system shall provide visual indicators (e.g., progress bars, checklists) to show the completion status of each milestone.
- The system shall notify users when a milestone is approaching its deadline.
- The system shall provide users with a dedicated section to manage their wish-list of potential projects they are interested in.
- Users shall be able to add, edit, and remove projects from their wish-list.
- The system shall allow users to save job postings or project opportunities to their wish-list for future reference.
- The system shall provide users with analytics on their ongoing projects, including:
    - Total number of projects in progress.
    - Percentage of projects completed on time.
- The system shall allow users to generate reports on their project tracking activities, including details about projects worked on, milestones achieved, and wish-list items.

Freelancer Proposal and Effort Estimation Configuration Module

- The system shall allow users to select and customize proposal templates that can be used for future job applications.
- The system shall enable users to save multiple proposal templates for different types of projects or clients.
- The system shall allow users to preview proposals before submitting them to clients.
- The system shall provide users with tools to estimate the time and effort required for various project types.
- Users shall be able to define parameters for effort estimation.
- The system shall allow users to set up predefined estimation criteria based on historical project data for improved accuracy.
- The system shall generate estimates based on user input and configuration settings, presenting them in an easily understandable format.

Freelancer Messages Suggestions Configuration Module

- The system shall provide automated message suggestions based on the context of the conversation with the client.
- The system shall analyze previous communications to recommend personalized message suggestions.
- Users shall have the option to customize suggestions before sending messages to clients.
- The users shall be able to configure the messages suggestions module.
- The system shall be able to generate responses while looking for the user account configuration for messages suggestions.

Freelancer Job Filtering and Spam Detection Configuration Module

- The system shall allow users to set filters for job postings based on criteria such as job type, budget, client rating, and deadlines.
- The system shall automatically identify and flag spam or irrelevant job postings using predefined keywords.
- Users shall be able to adjust spam detection parameters, such as frequency of certain keywords.

- The system shall allow users to manage spam detection keywords, including adding new keywords, editing existing ones, and removing outdated entries.
- The system shall enable users to extract key elements from job descriptions to facilitate better filtering and spam detection.
- Users shall have the ability to manage extracted keywords and phrases to enhance spam detection accuracy.

Freelancer Client Communication Management

- The system shall maintain a log of all communications between the freelancer and clients.
- Users shall be able to view the communication history for each client, accessible from the client's profile.
- The system shall provide a messaging interface for freelancers to send and receive messages from clients directly within the dashboard.
- Users shall be able to send text messages, attach files, and share links within the messaging interface.

Freelancer Notifications and Alerts Module

- The system shall allow users to manage their notification preferences, including the types of notifications they want to receive (e.g., project updates, messages, deadlines).
- Users shall be able to choose their preferred notification channels (e.g., email, in-app notifications).
- The system shall provide real-time alerts for important events, such as new messages from clients, job application updates, and upcoming deadlines.
- The system shall provide daily or weekly summary notifications, giving users an overview of missed messages, upcoming deadlines, and job opportunities.
- The system shall maintain a history of all notifications sent to users, allowing them to review missed alerts and actions taken.

Freelancer Reports Generation Module

- The system shall allow users to generate various types of reports, including project summaries, earnings reports.
- The system shall provide options for users to export reports in various formats, such as PDF, Excel, and CSV.
- Users shall be able to print reports directly from the dashboard.

Freelancer Dashboard and Analytics Module

- The system shall provide a customizable dashboard that displays key performance indicators (KPIs) and metrics relevant to the freelancer's work.
- The dashboard shall display real-time data on ongoing projects, earnings, client interactions, and proposal submissions using graphs, charts, and tables.
- The dashboard shall integrate alerts and notifications, allowing users to see important updates and reminders at a glance.
- The system shall ensure that the dashboard is user-friendly, with a clean layout and easy navigation.

Freelancer Bids Confirmation and Management Module (History)

- The system shall maintain a detailed history of all bids submitted by the freelancer, including the job title, bid amount, submission date, and status (e.g., pending, accepted, rejected).

- The system shall provide a confirmation step for bids before submission, allowing users to review their bid details and make any necessary adjustments.
- Users shall receive a confirmation notification upon successful submission of a bid.
- The system shall provide real-time status updates for each bid, notifying users of any changes in the bid's status (e.g., viewed by client, client decision made).
- Users shall be able to view detailed status information for each bid.

Freelancer Help and Support Module

- The system shall provide a centralized help center where users can access FAQs, guides, and troubleshooting articles.

Company Multi-Account Management Module

- The system shall allow users to add multiple Upwork accounts by providing authentication credentials for each account.
- Users shall be able to configure individual settings for each account, including notification preferences.
- The system shall provide a user-friendly interface to switch between different Upwork accounts seamlessly from a single dashboard.
- Users shall receive notifications indicating which account they are currently managing.
- The dashboard shall display an overview of all connected accounts, including key metrics such as active projects, bids submitted.
- The system shall allow admin users to set permissions and access levels for team members managing different Upwork accounts.
- Users shall be restricted from accessing accounts they do not have permission to manage.

Company Project Manager Module

- The system shall provide project managers with a dashboard to view all ongoing projects, including status, deadlines, and team assignments.
- Users shall be able to create, edit, and delete projects as needed.
- The system shall allow project managers to assign tasks to team members, set deadlines, and track task completion progress.
- Users shall receive notifications when tasks are assigned or updated.
- The system shall allow users to set and monitor project milestones, ensuring that key objectives are met within specified timeframes.
- Users shall receive alerts when milestones are approaching or overdue.

Company Internal-Organization Team Management Module

- The system shall enable admins to assign roles to team members, specifying their responsibilities and access levels within the team.
- Users shall be able to view and update role assignments as needed.
- The module shall include features for tracking team performance, including metrics related to project completion, deadline.
- Users shall be able to generate performance reports for individual teams and members.

Company Permissions and Access Level Management Module

- The system shall allow admins to define various user roles within the organization, specifying the permissions associated with each role.
- Users shall be able to view available roles and their permissions.

- The module shall provide granular access control, allowing admins to customize permissions for specific actions, features, or data at the user level.
- Users shall be restricted from accessing features or data not permitted by their assigned role.
- The system shall support dynamic role assignment, enabling admins to change user roles and permissions based on evolving organizational needs.
- Users shall receive notifications when their roles or permissions are updated.

Company Internal Organization Role Management Module

- The system shall allow admins to create and manage custom roles within the organization, specifying permissions and access levels for each role.
- Users shall be able to edit existing roles and remove them as necessary.
- The module shall enable admins to assign roles to individual users or groups, streamlining the process of managing access levels across the organization.
- Users shall receive notifications when they are assigned to a new role.
- The system shall provide predefined role templates based on common organizational structures, allowing for quick setup of roles.
- Admins shall be able to customize templates to meet specific organizational needs.

Company Kanban Module

- The system shall allow users to create and manage Kanban boards for tracking project tasks and workflows.
- Users shall be able to define columns representing different stages of the workflow (e.g., To Do, In Progress, Done).
- The module shall enable users to create tasks/cards within the Kanban board, assigning them to specific team members and setting deadlines.
- The module shall provide options for prioritizing tasks within the Kanban board, allowing users to set priority levels (e.g., High, Medium, Low).
- Users shall be able to filter tasks based on status, assignee, or due date for better visibility.
- The system shall include visual indicators of progress for each task, such as completion percentages or time tracking.

Proposal Generation Module

- The system shall allow users to input specific project details, including project scope, requirements, deadlines, and budget estimates.
- The module shall automatically generate proposals based on the provided inputs, ensuring that the content is coherent and relevant to the project requirements.
- The system shall incorporate predefined templates and formatting styles to maintain consistency across proposals.
- Users shall have the option to customize the generated proposal by adding personalized messages, additional sections, or specific content as needed.
- The system shall support text content extraction from various document formats (e.g., PDF, Word) to retrieve relevant information for proposal generation.
- The system shall allow users to create and customize proposal templates, including layout, sections.
- Users shall be able to utilize sections within templates for consistency across multiple proposals.
- The system shall allow users to define and estimate time requirements for each module within the proposal.

- The system shall allow users to access and retrieve information from previously completed projects to inform new proposals.
- The system shall support the automated insertion of relevant details from previous projects into the current proposal, enhancing the proposal's credibility and relevance.

Cost and Time Estimation Module

- The system shall allow users to define project modules (e.g., design, development, testing) based on the project's requirements.
- Users shall be able to set parameters for each module, including tasks, estimated duration, and resource allocation.
- The module shall provide input fields for users to enter estimated time for each defined task within the modules.
- Users shall be able to input cost factors for each module, including labor rates, material costs, and overhead expenses.
- The system shall calculate the total estimated cost for each module based on user inputs and predefined rates.
- The system shall allow users to retrieve historical data from previous projects to inform cost estimation for new projects.
- Users shall be able to filter past projects based on various criteria, such as project type, duration, and cost.
- The module shall automatically suggest cost estimates for new projects based on data from similar completed projects.
- Users shall have the option to customize these estimates based on the specific requirements of the new project.

Application Workflow Execution Module

- The system shall allow freelancers to initiate the purchase of connects through a dedicated interface.
- Users shall be able to view available packages of connects, including costs and quantities.
- The module shall integrate with a payment gateway to securely process transactions for connects.
- Users shall receive confirmation of successful transactions, including details of purchased connects.
- Users shall be able to view their current balance of connects in their dashboard.
- The module shall maintain a history of all connects purchases, including dates, amounts, and transaction statuses.
- Users shall be able to filter and search through their purchase history.
- The system shall provide clear error messages for failed purchases (e.g., insufficient funds, payment gateway issues).
- The system shall manage multiple workflows simultaneously (e.g., applying for jobs, purchasing connects) without interruption.
- The system shall provide a user-friendly interface for freelancers to apply for jobs.
- Users shall be able to select job postings and initiate the application process with a single click.
- The module shall support the automatic submission of pre-configured proposals for selected jobs.
- Users shall have the option to customize their proposals before submission.

- Users shall be able to track the status of their job applications (e.g., submitted, under review, accepted, rejected).
- The system shall provide notifications about any changes in application status.
- The module shall maintain a history of all job applications submitted by the user.
- The system shall provide an interface for freelancers to add projects to their profiles easily.
- Users shall be able to input project details, including title, description, skills required, and budget.
- The system shall automatically update the freelancer's profile with the newly added projects.
- Users shall receive confirmation once the project is successfully added to their profile.

SaaS Management Module

- The system shall allow users to subscribe to different service plans (e.g., free, basic, premium).
- Users shall be able to view their current subscription details, including plan type, start date, and renewal date.
- The system shall provide users with the ability to upgrade or downgrade their subscriptions at any time.
- Users shall receive notifications regarding the changes in subscription features and pricing when upgrading or downgrading.
- Users shall be able to view their billing history, including past invoices and payment statuses.
- Users shall have the ability to add, edit, or remove payment methods for their subscriptions.
- Users shall be able to cancel their subscriptions through the management interface.
- The system shall notify users of any cancellation policies and provide confirmation of cancellation.
- The system shall send notifications to users before their subscription renewal date to remind them of upcoming charges.
- Users shall receive alerts about any changes in subscription pricing or terms prior to renewal.

Data Management Module

- The system shall securely store user profile information, including name, email, contact number, and profile picture.
- Users shall be able to update their profile information, and the system shall reflect these changes in real-time.
- The system shall implement secure password storage using hashing and salting techniques.
- Users shall be able to reset their passwords via email verification and set new passwords through a secure interface.
- The system shall keep a comprehensive history of jobs applied for by users, including job details, application dates, and statuses.
- Users shall be able to view their jobs history through a user-friendly interface, with options to filter and search past applications.
- The system shall store information related to projects users are involved in, including project descriptions, milestones, and deadlines.
- Users shall have the ability to update project information and track progress through a dashboard.
- The system shall allow users to manage their preferences related to notifications, communication methods, and account settings.
- Preferences shall be stored securely and accessible for modification at any time.

- The system shall maintain role-based access control (RBAC) for users, specifying permissions based on assigned roles (e.g., super admin, individual freelancer, company freelancer).
- Administrators shall be able to assign or change user roles through the management interface.
- The system shall store generated proposals for users, allowing them to view and edit proposals before submission.
- Users shall have access to their proposal history, including details of submitted proposals and their statuses.
- The system shall allow administrators to define and manage parameters for identifying spam job postings.
- The system shall store all messages exchanged between users and clients.
- The system shall implement search functionality for users to find specific messages.

Non-Functional Requirements

Performance Requirements

- The system must respond to user requests within 4 seconds for management operations on bidbot, such as switching between dashboards, viewing data, and managing accounts information.
- The Upwork Data Scraping Module should efficiently handle scraping tasks for up to 50 accounts simultaneously, without affecting system performance.
- The system should support up to 500 concurrent users without degradation in performance.
- Proposals should be generated and displayed to the user within 10 seconds of submission.
- The cost and time estimation module should provide calculations within 5 seconds based on the data inputs.

Scalability Requirements

- The system must be scalable to handle thousands of users, including freelancers and companies, with their associated data, without performance degradation.
- The system should be able to handle and store large volumes of data, including scraped jobs, messages, and user profiles, growing at a rate of 10 thousand records per month.
- The system should be designed with a microservices architecture to allow the independent scaling of different modules (e.g., Data Scraping, Proposal Generation, Bidot Management).

Security Requirements

- The system must implement strong authentication mechanisms, including two-factor authentication (2FA), JWT tokens, and role-based access control (RBAC) for secure user access.
- The system must log and audit user activities, especially in sensitive areas like billing, account switching, and proposal submissions, to detect and prevent unauthorized access.
- The Upwork Data Scraping Module should have measures in place to avoid triggering security flags (e.g., CAPTCHA, IP blacklisting) while performing scraping operations.

Usability Requirements

- The dashboard interfaces (Freelancer, Company, Super Admin) must be intuitive and easy to use for both technical and non-technical users. The design should follow a simple and clean UI, ensuring ease of navigation.
- Both the Freelancer and Company dashboards must be accessible on mobile devices, with responsive design ensuring a seamless user experience on smartphones and tablets.
- The system should provide meaningful error messages and feedback when users encounter issues, such as form validation errors or failed operations.
- Users should be able to customize the UI color schemes, especially in the Super Admin and Freelancer Dashboards.

Reliability and Availability

- The system must guarantee 99.9% uptime to ensure that freelancers and companies can rely on it for their daily activities, particularly for time-sensitive tasks like job bidding.
- The system must ensure fault tolerance, especially in key modules like the Upwork Data Scraping, Proposal Generation, and Billing systems. It should have automated failover mechanisms for critical services.

Maintainability

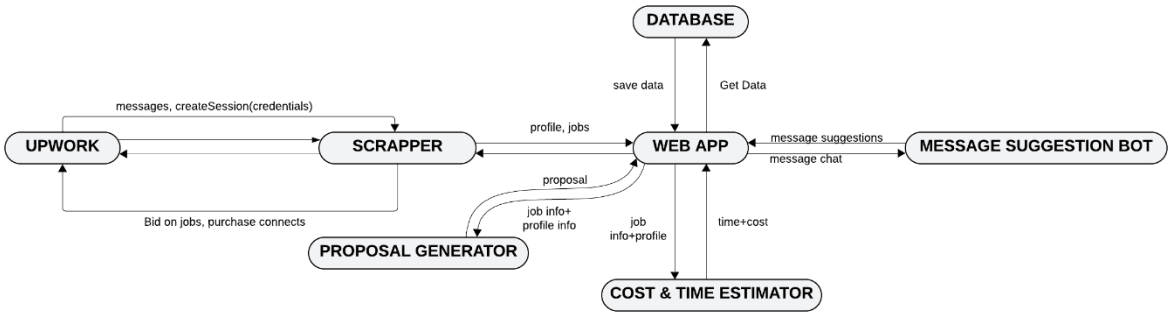
- The system should be designed with a modular architecture, making it easier to maintain, update, and extend individual modules (e.g., Proposal Generation, Data Management, Upwork Scraping) without affecting the entire system.
- All system code must follow standardized coding conventions and be well-documented to ensure ease of maintenance and collaboration among developers.

Compatibility Requirements

- The web application (Freelancer, Company, and Admin dashboards) must be compatible with all modern browsers, including Chrome, Firefox, Safari, and Edge, and support their latest versions.
- The system's API should follow RESTful standards and ensure compatibility with external systems that might integrate, such as payment gateways, and third-party tools.

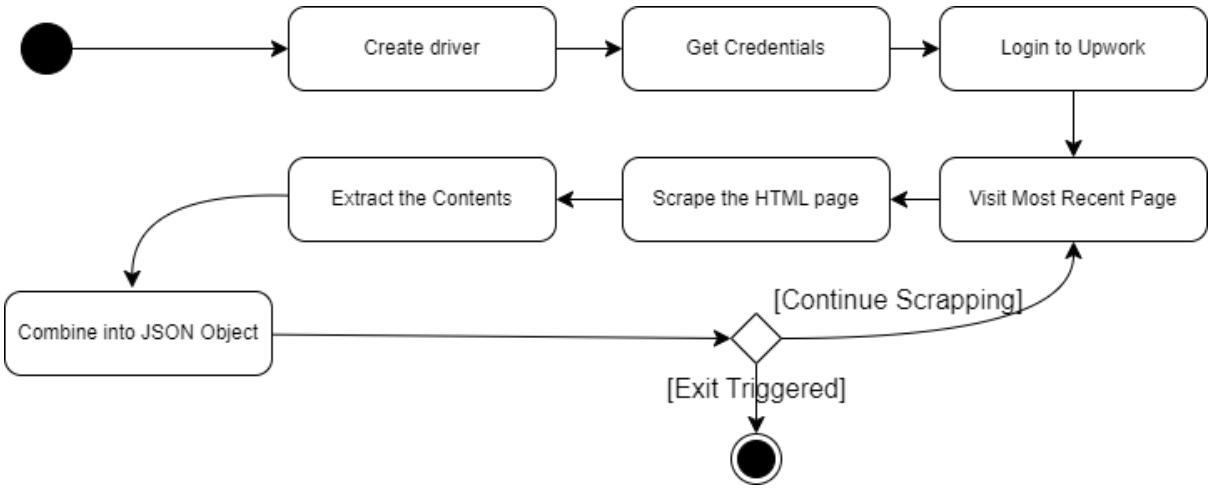
System Overview

Architecture Diagram

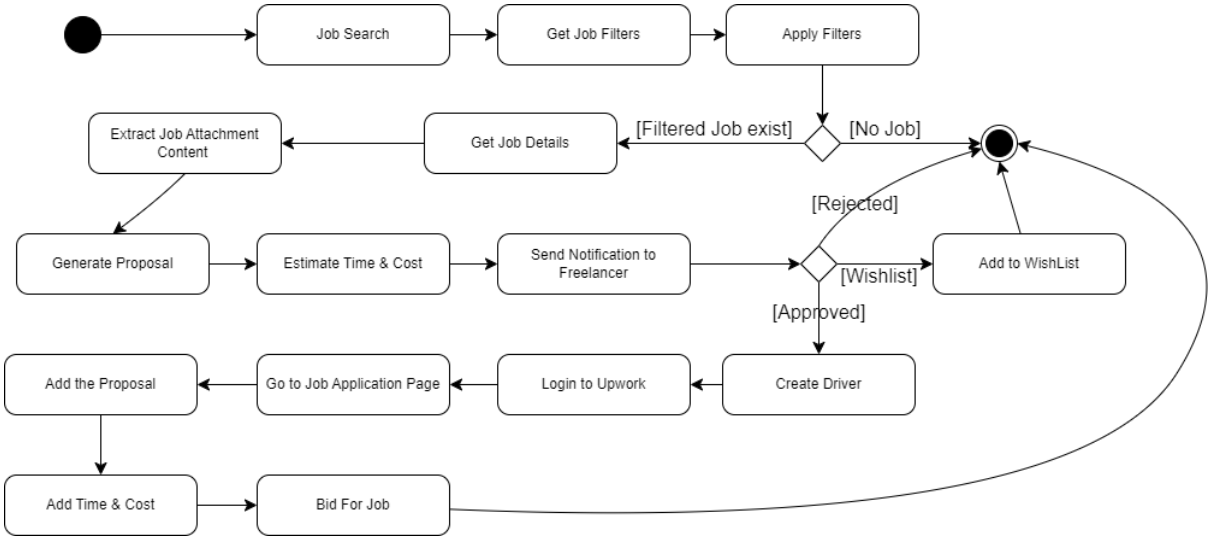


Activity Diagrams

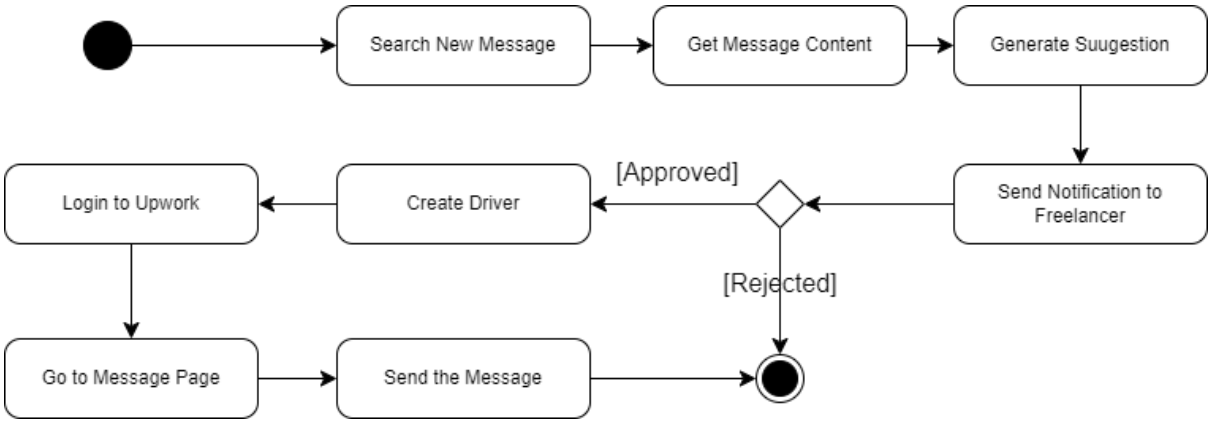
Job Search



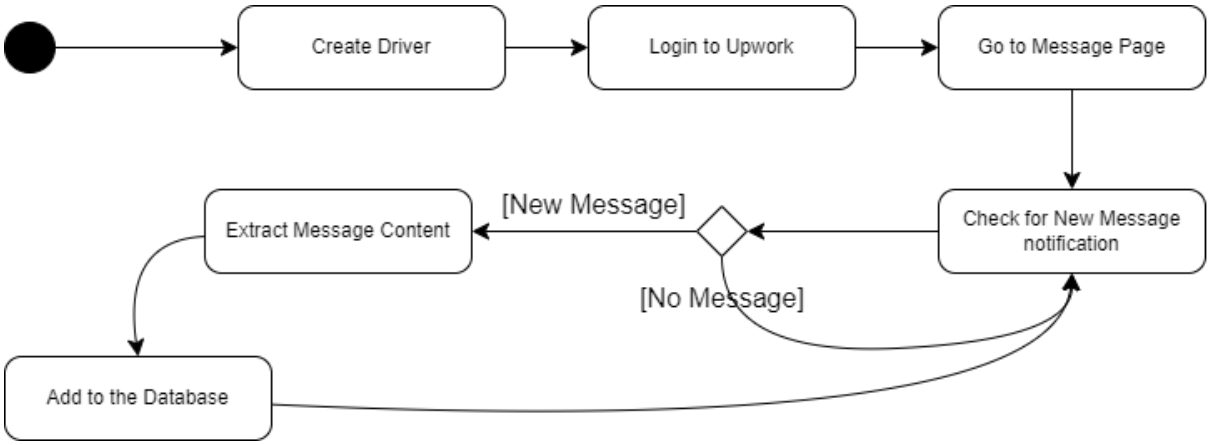
Job Apply



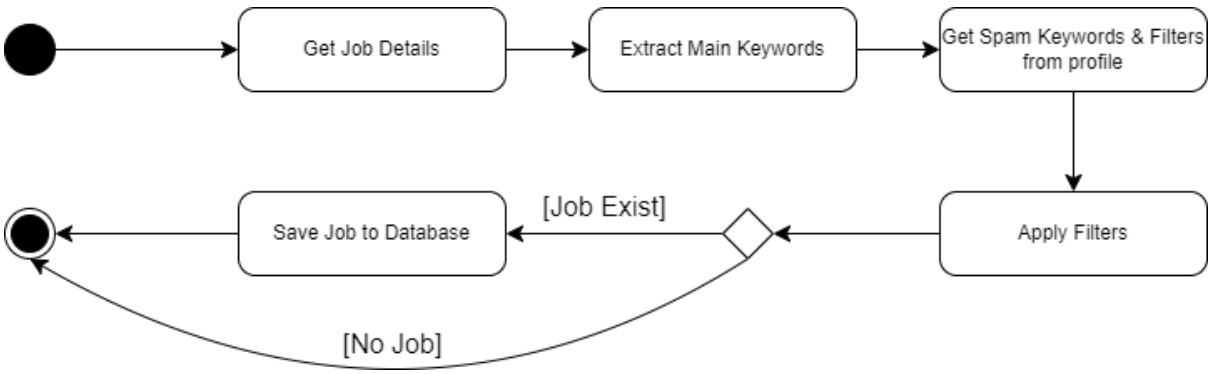
Send Message



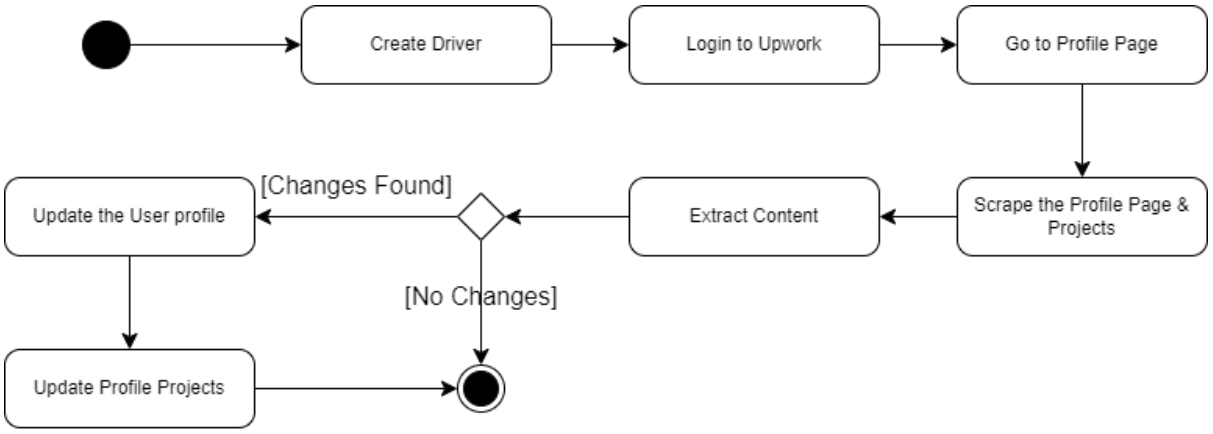
Search Message



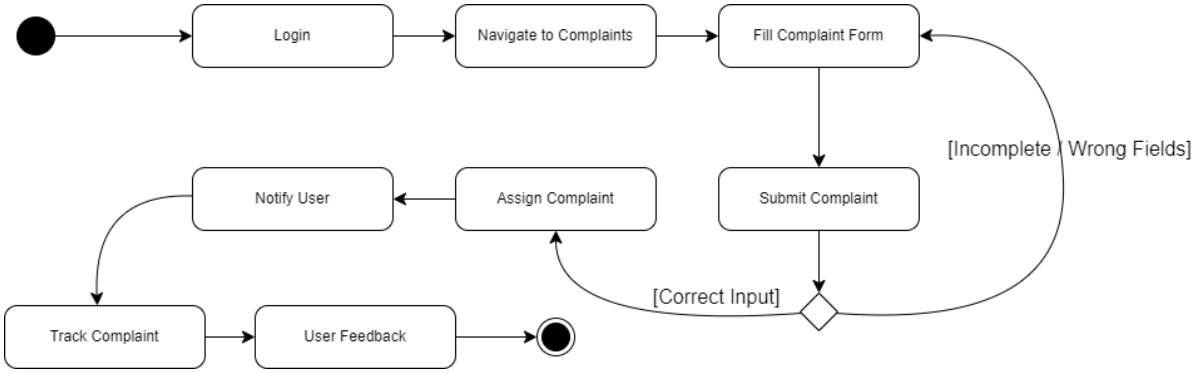
Job Filters



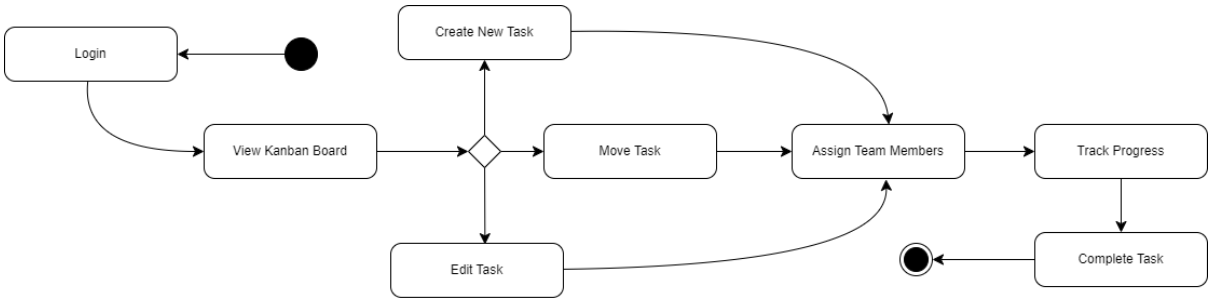
Update Profile



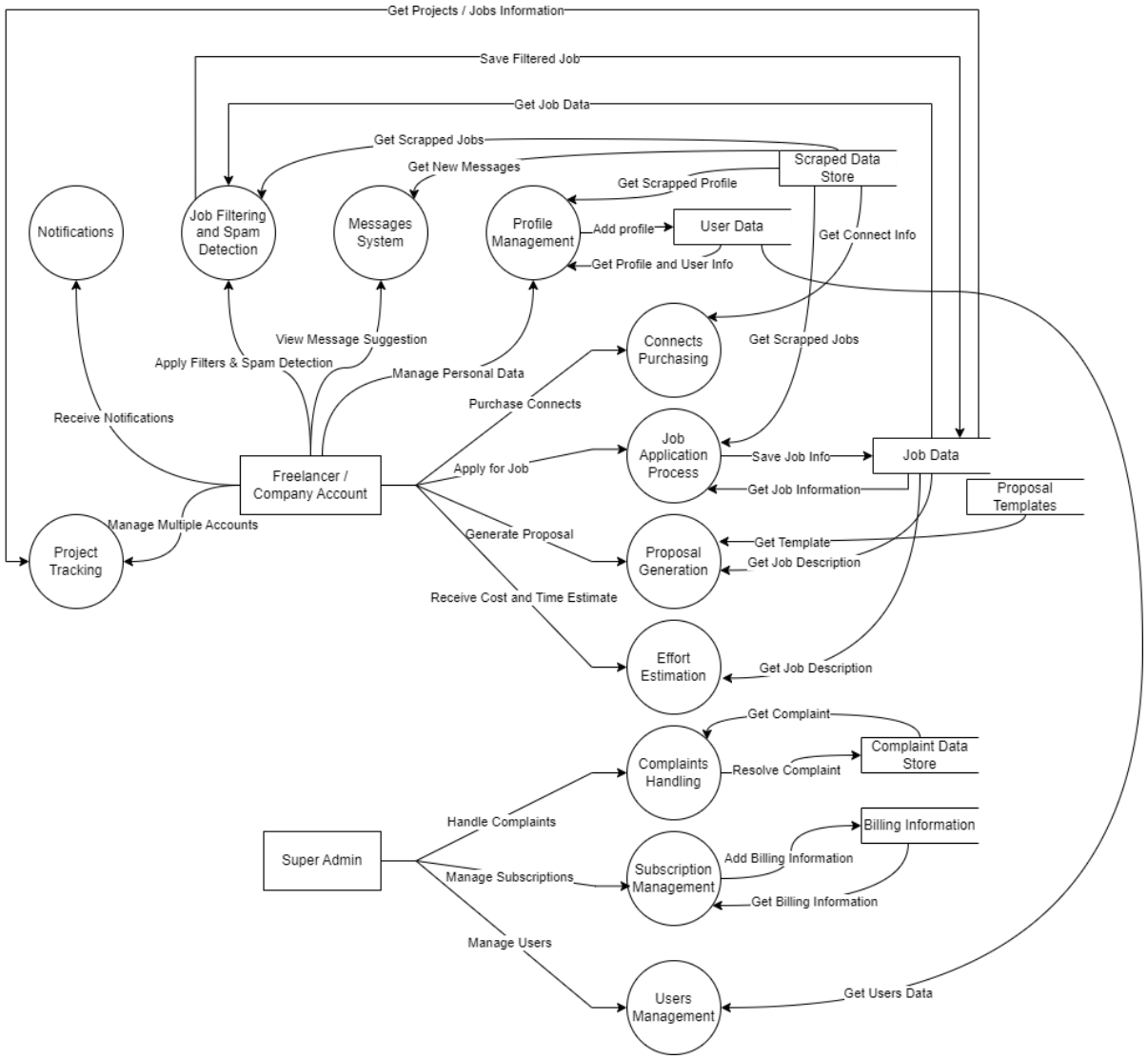
Complaints System



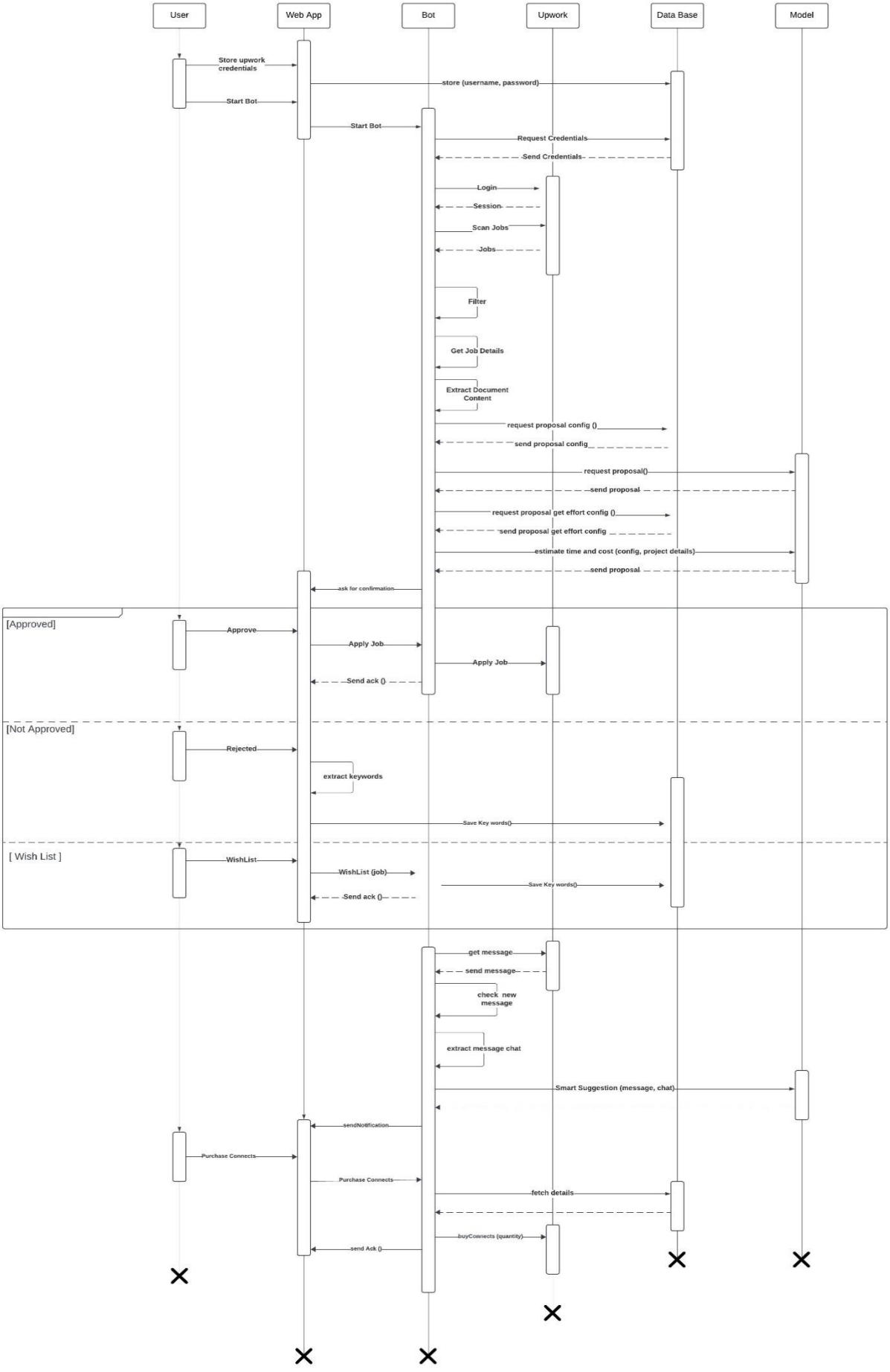
Kanban



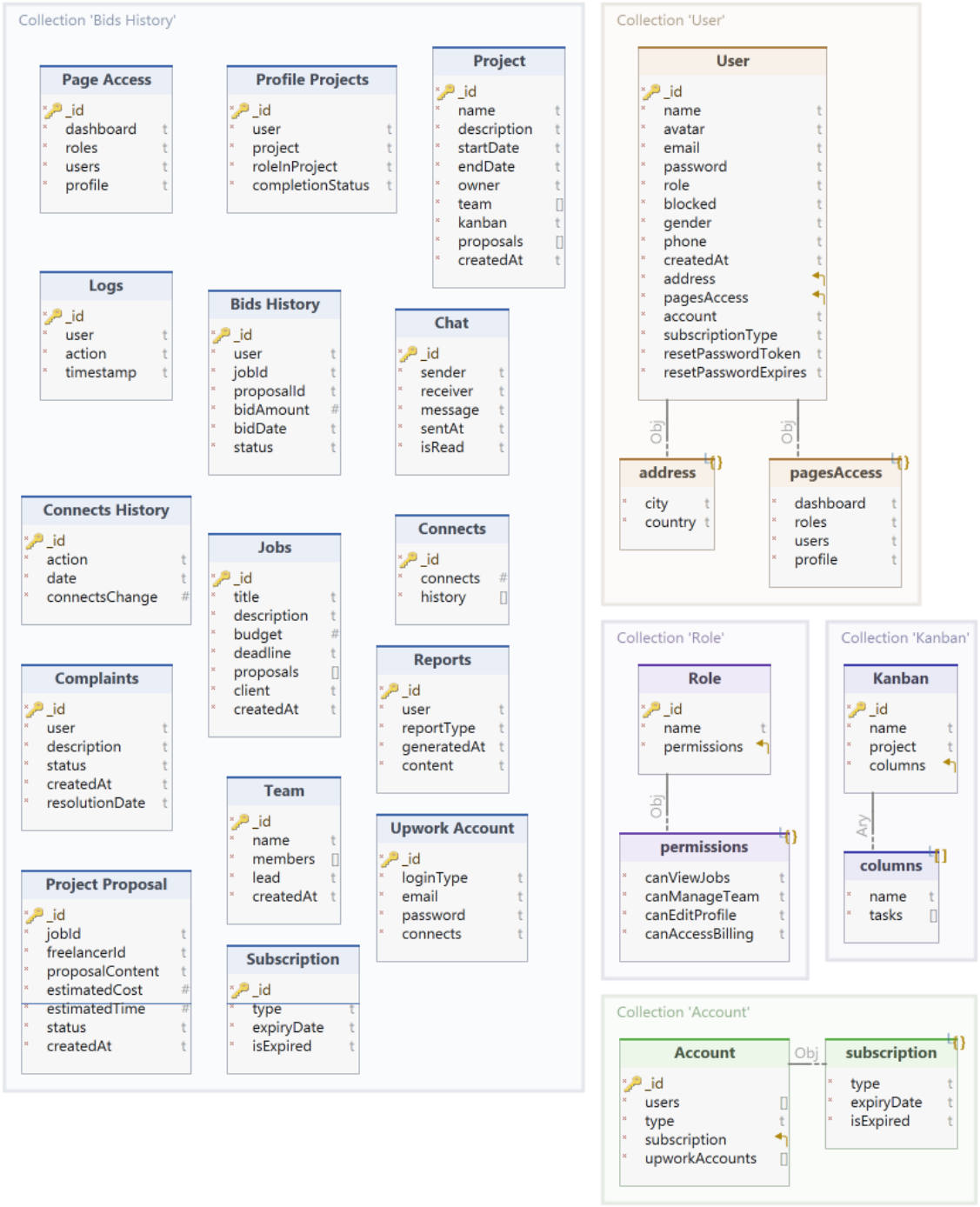
Data Flow Diagram



System Sequence Diagram



MongoDB Schema Diagram



MongoDB Schemas

Account Schema

```
const accountSchema = new Schema({
  users: [{ type: Schema.Types.ObjectId, ref: 'User' }],
  type: { type: String, enum: ['Company', 'Individual'], required: true },
  subscription: Subscription,
  upworkAccounts: [UpworkAccount]
});
```

Connects Schema

```
const connectsSchema = new Schema({
  connects: { type: Number, default: 0 },
  history: [{ type: Schema.Types.ObjectId, ref: 'ConnectsHistory' }]
});
```

Connects History Schema

```
const connectsHistorySchema = new Schema({
  action: { type: String, required: true },
  date: { type: Date, default: Date.now },
  connectsChange: { type: Number, required: true }
});
```

Subscriptions Schema

```
const subscriptionSchema = new Schema({
  type: { type: String, enum: ['Standard', 'Platinum', 'Gold'], required: true },
  expiryDate: { type: Date, required: true },
  isExpired: { type: Boolean, default: false }
});
```

Upwork Account Schema

```
const upworkAccountSchema = new Schema({
  loginType: { type: String, enum: ['emailPass', 'Google', 'Facebook'], required: true },
  email: { type: String, required: true },
  password: { type: String, required: true },
  connects: { type: Schema.Types.ObjectId, ref: 'Connects' }
});
```

Page Access Schema

```
const pageAccessSchema = new Schema({
  dashboard: { type: Boolean, default: true },
  roles: { type: Boolean, default: false },
  users: { type: Boolean, default: false },
  profile: { type: Boolean, default: false }
});
```

User Schema

```
const userSchema = new Schema({
  name: { type: String, required: true },
  avatar: { type: String, default: "https://cdn.pixabay.com/photo/2015/10/05/22/37/blank-profile-picture-973460_960_720.png" },
  email: { type: String, unique: true, required: true },
  password: { type: String, required: true },
  role: { type: String, enum: ["Individual Freelancer", "Company Freelancer", "Company Admin", "Admin", "Support Team", "None"], default: "None" },
  blocked: { type: Boolean, default: false },
  gender: { type: String, enum: ["Male", "Female", "Other"] },
  phone: { type: String},
  createdAt: { type: Date, default: Date.now},
  address: {
    city: { type: String },
    country: { type: String }
  },
  pagesAccess: pageAccessSchema,
  account: { type: Schema.Types.ObjectId, ref: 'Account' },
  subscriptionType: String,
  resetPasswordToken: String,
  resetPasswordExpires: Date
});
```

Project Schema

```
const projectSchema = new Schema({
  name: { type: String, required: true },
  description: { type: String },
  startDate: { type: Date },
  endDate: { type: Date },
  owner: { type: Schema.Types.ObjectId, ref: 'User' }, // Owner of the project
  team: [{ type: Schema.Types.ObjectId, ref: 'User' }], // Team members working on this project
  kanban: { type: Schema.Types.ObjectId, ref: 'Kanban' }, // Associated Kanban board
  proposals: [{ type: Schema.Types.ObjectId, ref: 'ProjectProposal' }], // Associated proposals
  createdAt: { type: Date, default: Date.now }
});
```

Kanban Schema

```
const kanbanSchema = new Schema({
  name: { type: String, required: true },
  project: { type: Schema.Types.ObjectId, ref: 'Project', required: true },
  columns: [{
    name: { type: String, required: true },
    tasks: [{ type: String }] // Task descriptions
  }]
});
```

Role Schema

```
const roleSchema = new Schema({
  name: { type: String, enum: ["Admin", "Support Team", "Freelancer", "Company Admin", "Company Member"], required: true },
  permissions: {
    canViewJobs: { type: Boolean, default: true },
    canManageTeam: { type: Boolean, default: false },
    canEditProfile: { type: Boolean, default: true },
    canAccessBilling: { type: Boolean, default: false }
  }
});
```

Profile Projects Schema

```
const profileProjectsSchema = new Schema({
  user: { type: Schema.Types.ObjectId, ref: 'User', required: true },
  project: { type: Schema.Types.ObjectId, ref: 'Project', required: true },
  roleInProject: { type: String, enum: ["Lead", "Contributor", "Viewer"], required: true },
  completionStatus: { type: String, enum: ["In Progress", "Completed", "Archived"], default: "In Progress" }
});
```

Project Proposal Schema

```
const projectProposalSchema = new Schema({
  jobId: { type: Schema.Types.ObjectId, ref: 'Jobs', required: true },
  freelancerId: { type: Schema.Types.ObjectId, ref: 'User', required: true },
  proposalContent: { type: String, required: true },
  estimatedCost: { type: Number, required: true },
  estimatedTime: { type: Number, required: true }, // In days
  status: { type: String, enum: ["Pending", "Approved", "Rejected"], default: "Pending" },
  createdAt: { type: Date, default: Date.now }
});
```

Chat Schema

```
const chatSchema = new Schema({
  sender: { type: Schema.Types.ObjectId, ref: 'User', required: true },
  receiver: { type: Schema.Types.ObjectId, ref: 'User', required: true },
  message: { type: String, required: true },
  sentAt: { type: Date, default: Date.now },
  isRead: { type: Boolean, default: false }
});
```

Bids History Schema

```
const bidsHistorySchema = new Schema({
  user: { type: Schema.Types.ObjectId, ref: 'User', required: true },
  jobId: { type: Schema.Types.ObjectId, ref: 'Jobs', required: true },
  proposalId: { type: Schema.Types.ObjectId, ref: 'ProjectProposal' },
  bidAmount: { type: Number, required: true },
  bidDate: { type: Date, default: Date.now },
  status: { type: String, enum: ["Submitted", "Accepted", "Rejected"], default: "Submitted" }
});
```

Jobs Schema

```
const jobsSchema = new Schema({
  title: { type: String, required: true },
  description: { type: String, required: true },
  budget: { type: Number, required: true },
  deadline: { type: Date },
  proposals: [{ type: Schema.Types.ObjectId, ref: 'ProjectProposal' }],
  client: { type: Schema.Types.ObjectId, ref: 'User' }, // The user posting the job
  createdAt: { type: Date, default: Date.now }
});
```

Team Schema

```
const teamSchema = new Schema({
  name: { type: String, required: true },
  members: [{ type: Schema.Types.ObjectId, ref: 'User' }],
  lead: { type: Schema.Types.ObjectId, ref: 'User', required: true },
  createdAt: { type: Date, default: Date.now }
});
```

Complaints Schema

```
const complaintsSchema = new Schema({
  user: { type: Schema.Types.ObjectId, ref: 'User', required: true },
  description: { type: String, required: true },
  status: { type: String, enum: ["Open", "Resolved", "Closed"], default: "Open" },
  createdAt: { type: Date, default: Date.now },
  resolutionDate: { type: Date }
});
```

Logs Schema

```
const logsSchema = new Schema({
  user: { type: Schema.Types.ObjectId, ref: 'User' },
  action: { type: String, required: true },
  timestamp: { type: Date, default: Date.now }
});
```

Reports Schema

```
const reportsSchema = new Schema({
  user: { type: Schema.Types.ObjectId, ref: 'User', required: true },
  reportType: { type: String, enum: ["Activity", "Billing", "Job Progress", "Team Performance"], required: true },
  generatedAt: { type: Date, default: Date.now },
  content: { type: String, required: true }
});
```